

Final-Year Project — Tech Stack Decisions

Locking down the open questions for the Multilingual SME Customer Service Chatbot

Project Owner

May 2026

Contents

Scope reminder	1
1. Hardware	2
1.1 Deployment infrastructure	2
1.2 Backup storage	2
2. Software — Backend	2
2.1 Async / queue	2
2.2 Orchestration	3
2.3 Multilingual model (embedding model)	3
2.4 Vector DB	4
3. Software — Frontend	4
3.1 Authentication	4
3.2 Charts (dashboard)	5
4. Cloud (hosting map)	5
5. Language model — fine-tuning decision	6
Why no fine-tuning for a final-year project	6
Repositioned thesis contribution	7
If the supervisor insists on a fine-tune	7
6. Channel integrations — only WhatsApp?	7
Channels in scope	7
Channels in Future Work (explicitly not built)	7
Why Meta Cloud API and not 360dialog / Twilio	8
7. Summary table (one-page reference)	8

Scope reminder

This is a **Master's final-year project**, not a commercial product. Decisions below optimise for:

1. **Demonstrable in a 30-minute panel defence** with 3–4 SME pilots.
2. **Buildable by one person** in ~6 months.
3. **Cheap** — student budget, free tiers wherever possible.
4. **Defensible academically** — clear rationale for every choice.

Anything that would be standard in a commercial deployment but adds weeks of work for no thesis value is deferred to **Future Work**.

1. Hardware

1.1 Deployment infrastructure

Decision: Single **cloud virtual machine** (managed PaaS), **not** dedicated SSD storage servers or bare-metal.

- **Backend API + worker:** one **Render** (or Fly.io) container instance running FastAPI + a small RQ worker process. ~\$7/month.
- **Database:** managed Postgres (Render Postgres free -> small tier, or Neon free tier).
- **Vector store:** Milvus Lite as a file on a persistent disk attached to the API container (no separate vector server).
- **Frontend:** Vercel Hobby tier (free).

Why not dedicated servers? For a pilot of 3–4 SMEs and ~12,000 messages/month, a single small container handles it. Self-managed servers introduce ops work that doesn't earn thesis marks.

1.2 Backup storage

The scope didn't specify backups but every credible system needs them, so document the plan:

Asset	Backup method	Retention
Postgres	Daily pg_dump -> Cloudflare R2	14 days
Vector DB (Milvus Lite file)	Weekly tar snapshot -> R2	30 days
SME-uploaded documents	R2's native versioning (always on)	30 days of versions
Audit log archives	Daily JSONL export -> R2	1 year

Cost: R2 storage at ≤5 GB total ≈ **\$0.08/month**.

2. Software — Backend

2.1 Async / queue

Decision: **Redis + RQ**. Not Celery.

Option	Verdict
RQ	Yes Lightweight, ~3 lines to enqueue, perfect for ingestion + webhook fan-out. Easy to defend in a thesis.
Celery	Powerful but heavy — multiple queue brokers, result backends, beat scheduler. Overkill at this scale and harder to explain.

Used for: document ingestion (chunk -> embed -> upsert), re-embedding on edit, daily analytics aggregation, escalation dispatch.

2.2 Orchestration

Decision: small custom finite-state machine. Not LangGraph or LangChain.

The conversation flow is six fixed steps (language detect -> retrieve -> prompt build -> LM call -> guards -> persist). Wrapping that in LangGraph adds a heavy dependency, a steeper learning curve, and ties the thesis story to a framework that will look different in a year.

A custom orchestrator is:

- **~100 lines of Python** — already specified in `02_CORE_ENGINE.md §8`.
- **Easier to explain to the panel** (“I composed five well-defined modules”) than to defend a framework choice.
- **Trivially testable** end-to-end without mocking a framework.

2.3 Multilingual model (embedding model)

Decision: sentence-transformers/all-MiniLM-L6-v2 for v1 (what the existing engine uses — keeps the 2 M-vector store intact). Upgrade path to `intfloat/multilingual-e5-base` documented for the thesis discussion section.

Model	Dim	Multilingual?	Pros	Cons
all-MiniLM-L6-v2	384	Trained on English	Yes Already used; fast on CPU; ~2 M vectors already embedded	Not formally multilingual; works for Pidgin only because the corpus is dense
multilingual-e5-base	768	Yes 100+ languages	Stronger on Yoruba/Hausa/Igbo	Forces re-embedding the 2 M corpus (real work)
multilingual-e5-large	1024	Yes	Even stronger	Larger memory, slower; overkill for thesis

Model	Dim	Multilingual?	Pros	Cons
BAAI/bge-m3	1024	Yes + dense+sparse+multi-vec	Top of class	Heaviest; out of scope for v1

Recommended thesis framing: “v1 ships with the existing English-trained embedder which empirically performs well on Pidgin due to corpus density; we report comparative retrieval-recall results against multilingual-e5-base on a 200-query held-out set as our ablation.”

2.4 Vector DB

Decision: **Milvus Lite** for v1 (already populated with 2 M vectors). Document **pgvector** as the production-scale upgrade.

Option	Verdict
Milvus Lite (file)	Yes Already in use, ~6 GB DB; single binary; no server to manage
pgvector	Compelling for production (unifies storage + tenancy in Postgres) — keep as Future Work upgrade path
Qdrant	Solid but adds another service to host
Weaviate	Heavier; over-engineered for this scope
Chroma	Simple but less performant on this size of corpus

3. Software — Frontend

3.1 Authentication

Decision: **Auth.js (NextAuth)**. Free, self-hosted, no vendor account required.

Option	Verdict
Auth.js	Yes Free; integrates natively with Next.js; supports Google OAuth + email magic link; sessions in your own Postgres
Clerk	Excellent UX, but adds a vendor account, free-tier message-count limits, and one more thing for the examiner to ask about
Supabase Auth	Great if you were already on Supabase — but you aren't; couples you to their stack

Magic-link email + Google OAuth covers every SME onboarding case.

3.2 Charts (dashboard)

Decision: Tremor.

- Tailwind-native, built for analytics dashboards, ships ready-to-style components (KPI cards, line/bar/area charts, donut, sparklines).
- Recharts is more flexible but requires more custom CSS work — irrelevant for a thesis demo.

4. Cloud (hosting map)

What	Where	Why
Backend API + worker (FastAPI)	Render (or Fly.io) — 1 small container	Cheapest managed option, persistent disk for Milvus, easy GitHub auto-deploy
Frontend (Next.js dashboard)	Vercel Hobby tier	Free; native Next.js host
Web widget bundle (widget.js)	Cloudflare Pages (or Vercel as a sub-route)	Static file, served at the edge
Postgres database	Render Postgres or Neon free tier	Managed, backed up, NDPR-friendly region available
Vector DB (Milvus Lite)	File on Render persistent disk attached to the backend container	Zero-ops; ~6 GB fits easily
Language model (LLM)	Groq API (Llama 3.3 70B Versatile, hosted by Groq)	No GPU rented, no model self-hosted — biggest cost saver
SME document uploads	Cloudflare R2 (S3-compatible, no egress fees)	~\$0.015 / GB-month; pre-signed URLs
Audit + backup archives	Same R2 bucket, separate prefix	One store to back up
Observability (LLM tracing)	Langfuse cloud free tier	Per-conversation traces
Error tracking	Sentry free tier	App-side errors
Uptime monitoring	UptimeRobot free	Pings the webhook and dashboard URLs

Total monthly cloud cost target: under \$15. All-up estimate at pilot volumes:

Item	Cost
Render backend (small)	~\$7
Render Postgres (small) or Neon (free -> small)	\$0–\$7
Vercel (Hobby)	\$0
Cloudflare R2	<\$1

Item	Cost
Groq API	usage-based; ~\$5–\$15/month at pilot volumes
Sentry / Langfuse / UptimeRobot	\$0
Domain name	\$1/month amortised
Total	≈ \$15–\$30/month

5. Language model — fine-tuning decision

Decision: No fine-tuning.

Verified against the existing engine (`ofofo_engine/config.py`), nothing has been fine-tuned. The system uses **Llama 3.3 70B Versatile hosted on Groq** through a thin client wrapper. Nigerian-language behaviour is achieved at **inference time** through three combined mechanisms, none of which touch model weights:

1. **Retrieval-Augmented Generation** over a 2-million-vector Nigerian corpus (Nairaland forum discourse, NaijaSenti tweets, English–Pidgin parallel pairs, Nigerian news, slang dictionaries).
2. **Detailed system prompts** that encode explicit Pidgin grammar rules (correct use of *am / e / im / dey / wetin*), forbidden translation patterns, and Nigerian public-figure context.
3. **Tone presets** (formal, casual, pidgin_friendly, youthful) per tenant.

Why no fine-tuning for a final-year project

Reason	Detail
Cost	A single 8B LoRA fine-tune + serving needs a GPU rental of ≈ \$250–\$300/month. The current Groq-hosted approach is ~\$10/month.
Time	Curation, training, evaluation, and serving adds 4–6 weeks of work that primarily proves something already established in the literature.
Risk	A bad fine-tune can degrade the model. The hosted general-purpose Llama 3.3 70B + your prompt engineering already produces strong Nigerian-fluent output (verified in the existing engine).
Thesis defensibility	The honest, novel contribution is the Nigerian corpus + RAG + prompt-engineering pipeline , not a fine-tune. This is a clean, well-bounded story.

Repositioned thesis contribution

The research contribution is a **Nigeria-specific, RAG-driven conversational pipeline** comprising: (a) a curated 2-million-vector Nigerian corpus, (b) a Pidgin-aware language detector that overrides off-the-shelf langid mistakes, (c) a per-tenant retrieval scheme with weighted document-type boosting, and (d) a tone-and-grammar prompt builder. The pipeline runs on top of a hosted general-purpose LLM (Llama 3.3 70B via Groq), and is evaluated against the same LLM **without** RAG, against GPT-4o, and against base Llama-3.1 baselines.

This is a **strong** Master's-level contribution. It tests three crisp ablations:

- *RAG vs no-RAG* (does the corpus help?)
- *Persona prompts vs vanilla prompts* (does our prompt engineering help?)
- *Pidgin-aware langid vs off-the-shelf langid* (a separate small-but-publishable result)

If the supervisor insists on a fine-tune

Run a small LoRA fine-tune of **Llama 3.1 8B** as a **Phase D extension** (Sprint 7–8 only), trained on the synthesised SME Q/A subset of the corpus, served via vLLM on a single RunPod L40S for the 2-week evaluation window only. Report as an ablation arm. Budget $\approx$ \$200 one-off.

6. Channel integrations — only WhatsApp?

Decision: WhatsApp Cloud API (Meta) + a first-party web widget. Instagram and Messenger are explicitly Future Work.

Channels in scope

Channel	Provider	Why
WhatsApp	Meta Cloud API (direct, official)	Free, no third-party fee; the 90%+ of Nigerian SMEs that matter. Embedded Signup flow handles tenant onboarding.
Web widget	First-party (your own JS bundle, no third party)	Required for panel demos — examiners can chat with the bot in the browser without giving you their phone numbers. ~3 days to build.

Channels in Future Work (explicitly not built)

Channel	Why out of scope
Instagram DMs	Same Meta Graph API stack as WhatsApp — adapter is ~3–5 days, but no thesis-novel content. Mention as Future Work.
Messenger	Same as above.
Telegram	Low Nigerian SME usage. Skip.
IVR / voice channel	Large undertaking; explicitly out of scope per the project brief.

Why Meta Cloud API and not 360dialog / Twilio

Provider	Cost	Setup	Verdict
Meta Cloud API (direct)	Free	Slower onboarding; need Meta Business verification	Yes Right for a thesis: free, official, longest-term option
360dialog	Paid (~\$30 + per-msg)	Easier onboarding	Use only as fallback if Meta verification stalls past Sprint 4
Twilio WhatsApp	Paid (per-msg)	Easy	Most expensive; avoid

7. Summary table (one-page reference)

Area	Decision
Cloud VM	Render small container (or Fly.io)
Backup	Daily pg_dump + R2 versioning
Async queue	Redis + RQ
Orchestration	Custom FSM (no LangGraph)
Embedding model	all-MiniLM-L6-v2 (already in use); document e5-base upgrade
Vector DB	Milvus Lite (file)
Auth (dashboard)	Auth.js (NextAuth)
Charts	Tremor
Backend hosting	Render
Frontend hosting	Vercel
LLM hosting	Groq API (Llama 3.3 70B)
File / upload storage	Cloudflare R2
Observability	Langfuse + Sentry + UptimeRobot
Fine-tuning	None — RAG + prompt engineering instead
Channels — built	WhatsApp (Meta Cloud API) + web widget
Channels — Future Work	Instagram, Messenger, voice
Languages — MVP	English + Pidgin (+ Yoruba as best-effort)

Area	Decision
Languages — Future Work	Full Hausa, full Igbo

This is the final, lockable answer to all seven open questions from PRD §10.